WEB-TO-WAREHOUSE

Product Intelligence Pipeline

DEPI Project Requirements & Implementation Checklist

Project	Web-to-Warehouse Product Intelligence Pipeline
Program	DEPI
Document	Requirements specification and deliverables checklist

1. Problem Statement

A retailer wants to continuously collect product information from permitted public web sources and combine it with its internal product catalog. Manual collection is slow and produces inconsistent records.

2. Project Objective

Build an automated data pipeline that extracts product information from APIs and/or permitted websites, standardizes and validates the records, detects duplicates, compares them with the internal catalog, and loads the resulting data into an analytical database.

3. Functional Requirements

- **FR-01 — Data ingestion:** Collect product data from public APIs and/or permitted websites. Use BeautifulSoup or Scrapy for website extraction where appropriate.
- **FR-02 — Responsible collection:** Check and respect robots.txt, rate limits, website terms, and any applicable access restrictions. Prefer official APIs when available.
- **FR-03 — Required fields:** Extract product name, category, price, rating, availability, and product URL. Record a source identifier and collection timestamp where possible.
- **FR-04 — Data cleaning:** Trim and normalize product names; standardize category labels; handle missing or malformed values; validate price, rating, availability, and URL fields.
- **FR-05 — Price normalization:** Normalize price formats and currencies where applicable. Preserve the original value and currency when conversion is performed, and document the conversion rate/source and timestamp.
- **FR-06 — Duplicate detection:** Identify duplicate products using stable source IDs where available, canonical URLs, and/or normalized names with category or other matching attributes.
- **FR-07 — Catalog matching:** Compare ingested records with the retailer's existing product catalog and match records to internal product IDs where possible.
- **FR-08 — Analytical storage:** Load cleaned data into PostgreSQL or MySQL using a repeatable ETL process. Avoid creating duplicate records during reruns.
- **FR-09 — Price history:** Store dated product-price snapshots so that historical prices can be queried and compared over time.

FR-10 — Change detection: Use SQL to identify price changes, newly discovered products, products no longer found, and category changes.

FR-11 — Orchestration: Schedule and orchestrate ingestion, transformation, validation, and loading with an Apache Airflow DAG.

FR-12 — Data quality: Add checks for required fields, valid data types, reasonable price/rating ranges, duplicate keys, and load completeness. Log failures and affected records.

FR-13 — Reporting: Create visualizations or a dashboard that summarizes product counts, price movements, availability, and detected catalog changes.

FR-14 — Documentation: Document setup, data sources, schema, pipeline steps, scheduling, data-quality rules, and how to run or troubleshoot the project.

4. Data Fields

Field	Description / rule
product_name	Required; cleaned and standardized text.
category	Category label standardized to a consistent format.
price	Numeric value; validated and normalized where possible.
currency	Currency code for the price, e.g. USD or EGP, when known.
rating	Numeric rating within the source's known scale; may be null if unavailable.
availability	Consistent status such as in_stock, out_of_stock, or unknown.
product_url	Canonical product URL; use as a matching signal where suitable.
source / source_product_id	Source name and source identifier, when available.
collected_at	Timestamp indicating when the product record was collected.

5. Suggested Database Design

The following is a suggested starting schema. Adapt names and constraints to the selected data sources and internal catalog.

Table	Purpose	Key fields (suggested)
products	Latest/current product record	product_id (PK), source, source_product_id, product_name, category, price, currency, rating, availability, product_url, updated_at
product_price_history	Historical price snapshots	snapshot_id (PK), product_id (FK), price, currency, captured_at
catalog_matches	Mapping between source products and internal catalog	source, source_product_id, internal_product_id, match_method, matched_at
pipeline_runs	Run status and audit information	run_id (PK), started_at, finished_at, status, records_read, records_loaded, error_summary

6. Required SQL Analysis

Prepare SQL queries or views to report the following:

- Products whose prices changed between snapshots, including previous price, current price, difference, and percentage change where possible.
- New products that appear in the latest ingestion but are not matched to the internal catalog or previous records.
- Products that were previously present but have not been observed in recent runs. Distinguish “not found” from confirmed removal unless the source provides reliable evidence.
- Products whose category differs from the previous record or internal catalog.

7. Pipeline Workflow

#	Step	What happens
1	**Extract**	Fetch data from an API or permitted website.
2	**Stage**	Save raw responses or raw records for traceability.
3	**Transform**	Clean fields, normalize formats, and standardize categories.
4	**Deduplicate & match**	Detect duplicate source records and match products to the internal catalog.
5	**Validate**	Run data-quality checks and log rejected records.
6	**Load**	Upsert current product records and append price snapshots.
7	**Analyze**	Run SQL change-detection queries and refresh visualizations.
8	**Orchestrate**	Schedule and monitor the steps using Airflow.

8. Expected Deliverables

Use this checklist to track project completion.

- ■ Scraper/API ingestion module
- ■ Database schema and SQL setup scripts
- ■ Historical product-price table
- ■ End-to-end ETL pipeline
- ■ Apache Airflow DAG
- ■ Data-quality checks and logging
- ■ SQL analysis for price/product/category changes
- ■ Visualization or dashboard
- ■ Project documentation and setup instructions

9. Optional Extension

Expose the latest product data through a REST API. Include documented endpoints, basic validation, and appropriate error responses.

10. Acceptance Criteria

- The pipeline can be run repeatedly without creating duplicate product records.
- The selected source is permitted for automated collection, and requests respect its access rules and rate limits.
- Required fields are extracted where the source makes them available; missing values are handled consistently.
- Cleaned records are loaded into PostgreSQL or MySQL with documented keys and relationships.
- Historical price snapshots are saved with timestamps and can be queried.
- SQL outputs identify price changes, new products, potentially missing/removed products, and category changes.
- The Airflow DAG defines the pipeline tasks and their dependencies, with run status visible.
- Data-quality failures are recorded and can be reviewed.
- A visualization and documentation are included in the final submission.

11. Suggested Project Structure

```
web-to-warehouse/
■■■ ingestion/ — API clients or scraper modules
■■■ transformations/ — cleaning, normalization, deduplication
■■■ database/ — schema and SQL scripts
■■■ dags/ — Airflow DAG definitions
■■■ tests/ — data-quality and pipeline tests
■■■ reports/ — SQL analysis and visualizations
■■■ README.md — setup and documentation
```

Prepared as a structured requirements document based on the supplied DEPI project brief.